University of Colorado Denver

Department of Computer Science & Engineering

CSCI/CSCY 4407: Security & Cryptography

Lab 5 Assignment: MAC,UF-CMA, CBC-MAC, HMAC & PRF

Instructor: Dr. Victor Kebande

Teaching Assistant: Celest Kester

Spring 2026

Assignment Instructions

This is a **group assignment**. Each group is required to complete and submit its own original work. You may discuss general concepts related to **message authentication codes (MACs)**, **UF-CMA security**, **CBC-MAC**, **HMAC**, and **pseudorandom functions (PRFs)** with classmates; however, **you are not permitted to share, reuse, or copy** solutions, mathematical derivations, code, scripts, screenshots, pseudocode, tag values, forged-message constructions, experimental observations, or final answers across groups.

For this lab, some tasks require you to **implement algorithms, generate your own test messages, keys, and experimental outputs**.

- **Implementation and experimentation tasks:** Each group must generate its own secret keys, input messages, message blocks, tags, and experimental results as instructed in the lab manual where necessary. This includes any code used to compute MACs, verify tags, simulate attacks, or compare constructions.
- **Analysis and attack tasks:** Each group must independently analyze the provided MAC constructions and adversarial settings, including forgery attempts, replay scenarios, or PRF-based reasoning, and document its own explanations and conclusions.

Because each group will use different messages, keys, and implementation choices, **tag values, verification outputs, attack observations, and intermediate results will differ across groups**. This is intentional and forms part of the assessment design.

It is your responsibility to ensure that you follow the lab instructions carefully and use only the files, parameters, examples, and artifacts assigned or generated for your own group. If any starter files, templates, or data files are provided, they will be available on Canvas under:

Modules → Lab 5: MACs and PRF Domain Extension

Unlike some previous labs, this assignment does **not use group-specific datasets**. Each group will generate its own:

- plaintext messages
- secret keys
- authentication tags
- experimental outputs

Groups must not copy scripts, outputs, screenshots, or explanations from other groups. All submitted work must reflect the group's own implementation and experiments.

All submitted work must clearly reflect your group's own implementation, analysis, screenshots, and explanations. Failure to follow these instructions may lead to grade penalties.

Academic Integrity and Collaboration

- **Allowed:** Discussing lecture material and cryptographic concepts at a high level.
- **Not allowed:** Sharing code, decrypted messages, keys, screenshots, or report content with other groups.
- **Not allowed:** Copying solutions from the internet, generative AI tools, previous students, or public repositories.
- **Submission:** Only **one** group member should submit the final report on Canvas on behalf of the group.

1 Introduction

Secure communication over open networks requires more than confidentiality. In many real-world systems, a receiver must also verify that a message truly originates from the claimed sender and that it has not been modified during transmission. These requirements correspond to the security properties of **integrity** and **authenticity**. Encryption alone does not guarantee these properties, which is why additional cryptographic mechanisms are needed.
A common misunderstanding in security is to assume that encryption by itself guarantees all aspects of protection. In reality, encryption is primarily intended to provide **confidentiality**, meaning that unauthorized parties should not learn the message contents. However,

confidentiality does not automatically imply that the received message is genuine or unmodified. An attacker who cannot read a message may still be able to tamper with it, replay an old message, replace one message with another, or trick a receiver into accepting a malicious command. For this reason, systems require mechanisms specifically designed to verify message authenticity and integrity.

One of the most important mechanisms for achieving integrity and authenticity is the **Message Authentication Code (MAC)**. A MAC is a keyed function shared between a sender and a receiver. Given a secret key K and a message M, the sender computes an authentication tag

$$T = T_K(M)$$

and sends the pair (M, T) to the receiver. The receiver recomputes the tag using the same key and verifies that

$$T_K(M) = T$$

If the values match, the message is accepted as authentic; otherwise it is rejected. In this way, MACs allow the receiver to detect unauthorized modifications and verify the origin of the message.

The security of MACs is often defined using the notion of **Unforgeability under Chosen Message Attack (UF-CMA)**. Informally, this means that even if an adversary can request valid tags for many messages of its choice, it should still be computationally infeasible for the adversary to generate a valid tag for a *new* message. This security notion models realistic attackers that interact with authentication systems before attempting to forge messages.

Many MAC constructions are based on block ciphers. One example is the **CBC-MAC**, where a message is divided into blocks

$$M = M[1] \| M[2] \| \cdots \| M[m]$$

and processed iteratively using a block cipher E_K. The chaining computation can be expressed as

$$C[i] = E_K(C[i-1] \oplus M[i]), \quad C[0] = 0^n$$

and the final authentication tag is

$$T = C[m].$$

Although CBC-MAC appears secure, improper use can lead to attacks such as message splicing or forgery. These weaknesses demonstrate that designing secure MAC constructions requires careful analysis.

Another closely related cryptographic primitive is the **Pseudorandom Function (PRF)**. A PRF is a keyed function that behaves like a random function to any efficient adversary that does not know the key. Formally, a PRF is a function family

$$F : \{0,1\}^k \times D \to \{0,1\}^n$$

that is computationally indistinguishable from a truly random function. PRFs are important because they can be used to construct secure MACs and other cryptographic mechanisms.
In practice, secure MAC constructions have been standardized to address weaknesses in simpler approaches. Examples include **CMAC** and **HMAC**. HMAC in particular is widely used in protocols such as TLS, SSH, and IPsec. It uses a cryptographic hash function H and two derived keys to compute

$$\mathrm{HMAC}(K, M) = H\big((K \oplus opad) \,\|\, H((K \oplus ipad) \,\|\, M)\big).$$

This lab introduces the principles behind message authentication codes and their relationship with pseudorandom functions. Through analysis and experimentation, students will explore how MACs protect message integrity, why some constructions are insecure, and how modern cryptographic designs such as HMAC provide stronger security guarantees.

2 Assignment Objectives

The objective of this assignment is to develop a practical and theoretical understanding of **Message Authentication Codes (MACs)** and their relationship to **Pseudorandom Functions (PRFs)**. Students will explore how cryptographic authentication mechanisms protect message integrity and authenticity, and how secure constructions are designed and evaluated.
Through analysis, implementation, and experimentation, this assignment will help students understand how authentication tags are generated and verified, how adversaries may attempt to forge messages, and why certain constructions are secure while others are not.
By completing this assignment, students will be able to:

- Understand the fundamental purpose of **message authentication** in secure communication systems.
- Explain how a **Message Authentication Code (MAC)** is computed and verified using a shared secret key.
- Apply the mathematical formulation of MAC computation, such as

  $$T = T_K(M)$$

  and understand the verification condition

  $$T_K(M) = T.$$

- Understand the concept of **Unforgeability under Chosen Message Attack (UF-CMA)** and why it is an important security definition for MAC schemes.
- Analyze how block-cipher-based constructions such as **CBC-MAC** compute authentication tags using iterative block operations.

- Recognize structural weaknesses in naive cryptographic constructions and understand how adversaries may attempt to exploit them.
- Understand the concept of a **Pseudorandom Function (PRF)** and how it relates to message authentication mechanisms.
- Explore how secure constructions such as **HMAC** use cryptographic hash functions and key transformations to produce robust authentication tags.
- Gain practical experience implementing authentication algorithms, verifying message integrity, and analyzing security properties.
- Develop the ability to interpret experimental results and explain how authentication mechanisms protect against message forgery and tampering.

3 Lab Requirements

To successfully complete this lab, students must prepare a suitable environment for performing cryptographic experiments related to **Message Authentication Codes (MACs)** and **Pseudorandom Functions (PRFs)**. The tasks in this assignment require students to generate messages, compute authentication tags, analyze authentication mechanisms, and execute scripts within a Linux-based system.

All experiments for this lab must be conducted in a **Linux environment**. Students are required to use one of the following platforms:

- **Kali Linux** (recommended)
- **Ubuntu Linux**
- A university lab workstation running Linux
- A Linux virtual machine using **VirtualBox** or **VMware**

3.1 Required Software Tools

The following tools will be required to complete the lab tasks:

- **Python 3** – Used to implement MAC algorithms and perform authentication experiments.
- **Python cryptographic libraries** – Built-in modules such as `hashlib` and `hmac` will be used to compute message authentication codes.
- **Text editor or code editor** – Examples include `nano`, `vim`, `gedit`, or `VS Code`.
- **Linux terminal** – Used to execute commands, run scripts, and capture the outputs required for documentation.

3.2 Required Installations

Before beginning the lab tasks, students should ensure the required tools are installed by running the following commands in the terminal:

```
sudo apt update
sudo apt install python3 python3-pip openssl
```

To confirm that Python is installed correctly, run:

```
python3 --version
```

3.3 Working Directory Setup

Students should create a dedicated working directory for the lab where all files, scripts, and results will be stored. For example:

```
mkdir MAC_Lab
cd MAC_Lab
```

All scripts, plaintext messages, generated tags, and experimental outputs should be stored in this directory.

Documentation Requirements

Students must carefully document their work throughout the lab. The final submission must include:

- Screenshots showing the commands executed in the terminal.
- Python scripts used to generate or verify authentication tags.
- Generated MAC values and verification outputs.
- Short explanations describing the purpose and outcome of each task.

All screenshots must clearly show the command used and the resulting output. These screenshots will form part of the required evidence in the lab report submitted through Canvas.

4 Lab Tasks

This lab will be completed step by step. In this assignment, you will implement and test basic message authentication mechanisms, observe how authentication tags are generated and verified, and analyze why secure constructions are needed in practice.

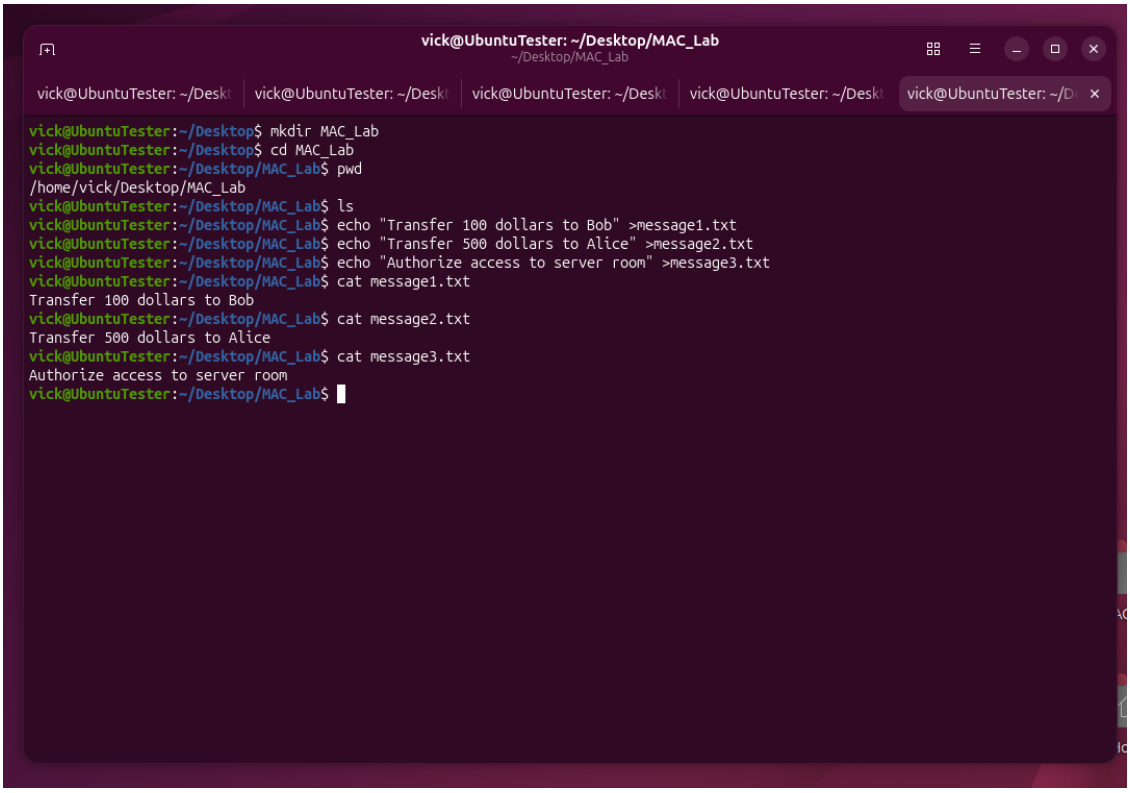

Figure 1: Example screenshot showing the Lab Working Directory and test environment

4.1 Task 1: Create the Lab Working Directory

Begin by creating a dedicated folder for this lab. This folder will store all scripts, text files, and screenshots required for submission as is shown in Figure 1

```
mkdir MAC_Lab
cd MAC_Lab
```

After creating the folder, confirm that you are inside the correct directory:

```
pwd
ls
```

What to do:

- Open your Kali Linux or Ubuntu terminal.
- Create the lab directory.
- Navigate into the directory.
- Verify the directory location.

4.2 Task 2: Create and Inspect Sample Message Files

Create plaintext message files that will be used for the authentication experiments in later tasks. These files will represent messages that a sender may transmit to a receiver.
Create three different message files using the terminal:

```
echo "Transfer 100 dollars to Bob" > message1.txt
echo "Transfer 500 dollars to Alice" > message2.txt
echo "Authorize access to server room" > message3.txt
```

Step 1: Display the File Contents
Verify that the messages were created correctly.

```
cat message1.txt
cat message2.txt
cat message3.txt
```

Step 2: Inspect File Information
Check the size of the files using:

```
ls -l message*.txt
```

Record the file sizes and compare them.

Step 3: Compute Message Hashes
Compute the SHA-256 hash of each message file:

```
sha256sum message1.txt
sha256sum message2.txt
sha256sum message3.txt
```

Record the resulting hash values.

Step 4: Modify a Message
Modify one of the files:

```
echo "Transfer 1000 dollars to Bob" > message1.txt
```

Compute the hash again:

```
sha256sum message1.txt
```

Observe how a small change in the message results in a completely different hash value.

What to Submit

- Screenshot showing the creation of the message files.
- Screenshot showing the contents of the files using `cat`.
- Screenshot showing file sizes using `ls -l`.
- Screenshot showing the SHA-256 hashes of the files.
- A short explanation describing how a small modification changes the hash value.

4.3 Task 3: Write and Test a Basic HMAC Generation Script

Create a Python script that computes an HMAC tag for a message using a shared secret key. This demonstrates the basic MAC idea where a sender computes a tag

$$T = T_K(M)$$

for a message M using a secret key K.
Create the script:

```
nano hmac_gen.py
```

Write the following code:

```
import hmac
import hashlib

key = b"secretkey123"
filename = "message1.txt"

with open(filename, "rb") as f:
    message = f.read()

tag = hmac.new(key, message, hashlib.sha256).hexdigest()

print("Message:", message.decode())
print("HMAC Tag:", tag)
```

Save the file and run it:

```
python3 hmac_gen.py
```

What to do:

- Create the Python script exactly as shown and save it as `hmac_gen.py`.
- Run the script and record the generated HMAC tag for `message1.txt`.
- Explain what the script is doing step by step.
- Modify the value of `filename` so that the script computes HMAC tags for `message2.txt` and `message3.txt`.
- Record all generated HMAC tags and compare them.
- Explain why different messages produce different HMAC values even when the same secret key is used.
- Modify one of the message files slightly and compute its HMAC again.
- Compare the old and new tag values and explain what this reveals about message integrity.

4.4 Task 4: Verify the Authenticity of a Message

In the previous task, you generated an HMAC authentication tag for a message. In this task, you will implement a verification script that checks whether a received message and tag are valid.
The receiver accepts the message only if

$$T_K(M) = T$$

where T is the received tag and $T_K(M)$ is the tag computed by the receiver using the same secret key.

Step 1: Create the Verification Script
Create a new Python file:

```
nano hmac_verify.py
```

Write the following code:

```
import hmac
import hashlib

key = b"secretkey123"
filename = "message1.txt"
provided_tag = input("Enter the received HMAC tag: ").strip()

with open(filename, "rb") as f:
    message = f.read()

computed_tag = hmac.new(key, message, hashlib.sha256).hexdigest()

if hmac.compare_digest(provided_tag, computed_tag):
    print("Verification successful: Message is authentic.")
else:
    print("Verification failed: Message has been altered or tag is invalid.")
```

Save the script and run it:

```
python3 hmac_verify.py
```

Enter the correct tag generated in **Task 3**.

Step 2: Test Successful Verification
Run the script using the correct HMAC tag and observe the output. The program should confirm that the message is authentic.

Step 3: Test Verification with an Incorrect Tag
Run the verification script again but modify the tag slightly (for example change one or two characters). Observe the result.

Step 4: Modify the Message
Edit the message file:

```
nano message1.txt
```

Change the message slightly (for example modify the amount in the transfer). Save the file and run the verification script again using the original tag generated in Task 3.
Observe the result.

Step 5: Repeat the Experiment with Another Message
Repeat the verification experiment using `message2.txt` and its corresponding tag from Task 3.

What to do:

- Run the verification script using the correct HMAC tag and record the successful verification result.
- Run the verification script using an incorrect tag and record the failure result.
- Modify the message file and run the verification script again using the original tag.
- Observe and explain why the verification fails after the message is modified.
- Repeat the verification experiment for another message file.
- Write a short explanation describing how HMAC verification ensures message integrity and authenticity.

4.5 Task 5: Modify the Message and Test Integrity Failure

In this task, you will simulate a **message tampering attack**. An attacker may attempt to modify the contents of a message after it has been authenticated. The purpose of a MAC is to ensure that such modifications are detected during verification.

Step 1: Modify the Message
Modify the contents of `message1.txt`. For example:

```
echo "Transfer 9000 dollars to Eve" > message1.txt
cat message1.txt
```

Step 2: Run the Verification Script
Run the verification script again using the **original tag generated in Task 3**.

```
python3 hmac_verify.py
```

Enter the same tag that was previously valid.

Step 3: Observe the Result
The verification should now fail because the message content has changed while the authentication tag remains the same.

Step 4: Try Additional Tampering
Perform at least two additional modifications to the message, for example:

```
echo "Transfer 100 dollars to Eve" > message1.txt
```

or

```
echo "Transfer 100 dollars to Bob immediately" > message1.txt
```

Run the verification script again after each modification and observe the result.

What to observe:

- The verification should fail whenever the message content is modified.
- Even small changes in the message should result in a completely different HMAC value.
- The receiver can detect that the message was altered.

What to do:

- Modify the message file at least three different times.
- Run the verification script after each modification using the original tag.
- Record the verification results.
- Explain why the authentication tag becomes invalid after the message is changed.
- Describe how this experiment demonstrates the role of MACs in protecting message integrity.

4.6 Task 6: Generate HMAC Tags for Multiple Messages

In this task, you will generate HMAC authentication tags for multiple messages using the same secret key. This experiment demonstrates that even when the same key is used, different messages produce different authentication tags.

Step 1: Create a Script for Multiple Messages
Instead of computing the HMAC for a single message, modify the script so it processes several message files automatically.
Create a new script:

```
cp hmac_gen.py hmac_multi.py
nano hmac_multi.py
```

Replace the contents with the following code:

```
import hmac
import hashlib

key = b"secretkey123"
files = ["message1.txt", "message2.txt", "message3.txt"]

for filename in files:
    with open(filename, "rb") as f:
        message = f.read()

    tag = hmac.new(key, message, hashlib.sha256).hexdigest()

    print("File:", filename)
    print("Message:", message.decode())
    print("HMAC Tag:", tag)
    print("-" * 60)
```

Step 2: Run the Script
Execute the script:

```
python3 hmac_multi.py
```

Observe the authentication tags generated for each message.

Step 3: Modify One Message
Edit one of the message files slightly:

```
nano message2.txt
```

Change a small part of the message (for example modify a number or word). Run the script again:

```
python3 hmac_multi.py
```

Observe how the HMAC tag changes even though the modification was small.

Step 4: Compare Results
Compare the authentication tags produced for all messages before and after modification.

What to observe:

- Each message produces a different HMAC tag even though the same secret key is used.
- A small modification in the message results in a completely different authentication tag.
- This behavior helps protect message integrity.

What to do:

- Generate HMAC tags for all three message files.
- Record the resulting tags.
- Modify one message and recompute its HMAC tag.
- Compare the old and new tag values.
- Explain why different messages produce different authentication tags.
- Explain how this property helps prevent message forgery.

4.7 Task 7: Change the Secret Key and Observe the Effect

In the previous tasks, you generated HMAC tags using a fixed secret key. In this task, you will investigate how the authentication output changes when the secret key is modified.
The security of a Message Authentication Code depends on the secrecy of the key. If the key changes, the authentication tags should also change even when the message remains the same.

Step 1: Modify the Secret Key
Open your HMAC generation script:

```
nano hmac_multi.py
```

Locate the line defining the secret key:

```
key = b"secretkey123"
```

Change it to a different value, for example:

```
key = b"newsecret456"
```

Save the file.

Step 2: Run the Script Again
Execute the script again:

```
python3 hmac_multi.py
```

Observe the newly generated HMAC tags.

Step 3: Compare Results
Compare the HMAC tags produced with the original key and the new key.

Step 4: Try Another Key
Modify the key one more time (for example create your own secret key) and run the script again.

What to observe:

- Changing the secret key results in completely different HMAC tags even when the message remains unchanged.
- The authentication tag depends on both the message and the secret key.
- Without knowing the secret key, an attacker cannot generate the correct authentication tag.

What to do:

- Change the secret key and recompute the HMAC tags.
- Record the tags generated using the new key.
- Compare the old and new tag values.
- Repeat the experiment using another key of your choice.
- Explain why the authentication output depends on the secret key.
- Describe why keeping the key secret is essential for message authentication.

4.8 Task 8: Replay Attack Experiment

In the previous tasks, you generated valid authenticated message–tag pairs using HMAC. While a MAC provides **integrity** and **authenticity**, it does not automatically provide **freshness**. This means that an attacker may capture a valid message and its authentication tag, then send the exact same pair again at a later time. This is called a **replay attack**.
In this task, you will simulate such a replay attack step by step and observe why a system that only checks the MAC may still accept the repeated message.

Step 1: Restore the Original Message
Make sure `message1.txt` contains the original valid message.

```
echo "Transfer 100 dollars to Bob" > message1.txt
cat message1.txt
```

You should see the message displayed in the terminal.

Step 2: Generate a Valid Authentication Tag
Run your HMAC generation script to compute a valid authentication tag for the message.

```
python3 hmac_gen.py
```

The script will display the message and its HMAC tag. Record the tag shown in the output. This forms the valid authenticated pair:

$$(M, T)$$

where M is the message and T is the corresponding tag.

Step 3: Verify the Original Message
Run the verification script:

```
python3 hmac_verify.py
```

When prompted, paste the HMAC tag generated in Step 2 and press Enter.
The expected output should indicate that verification was successful.

Step 4: Simulate the Replay Attack
Now assume an attacker captured the valid message and its correct tag from the previous communication. The attacker does not change the message and does not create a new tag. Instead, the attacker simply sends the **same message and same tag again**.
Run the verification script a second time:

```
python3 hmac_verify.py
```

When prompted, enter the **same HMAC tag** again.
Observe that the message is accepted again. This demonstrates replay: the receiver accepts the repeated message because the MAC is still valid.

Step 5: Repeat the Replay Once More
To confirm the behavior, run the same verification process again using the same message and the same tag:

```
python3 hmac_verify.py
```

Enter the same tag once more.
If the message is accepted again, this shows that the verification process checks correctness of the tag, but it does not know whether the message has already been processed before.

Step 6: Record and Compare the Results
For this step, record the outcome of the following three checks:

- First verification of the valid message
- Second verification using the same message and same tag
- Third verification using the same message and same tag

You should observe that all three verifications succeed as long as the message and tag remain valid.

Step 7: Explain What Happened
Based on your experiment, answer the following:

- What happens when the same authenticated message (M, T) is verified multiple times?
- Why does the receiver still accept the replayed message?
- Why does a MAC alone not guarantee freshness?

Step 8: Add a Timestamp or Counter
To reduce replay attacks, many systems include a value such as a timestamp, sequence number, or counter inside the authenticated message.
Modify `message1.txt` to include a timestamp:

```
echo "Timestamp: 1700000000 | Transfer 100 dollars to Bob" > message1.txt
cat message1.txt
```

Step 9: Generate a New Tag for the Updated Message
Since the message has changed, generate a new HMAC tag:

```
python3 hmac_gen.py
```

Record the new tag.

Step 10: Verify the Updated Message
Run the verification script again:

```
python3 hmac_verify.py
```

Enter the new tag generated in Step 9.
Observe that the updated message is accepted only when the correct new tag is used.

Step 11: Discuss Replay Prevention
Write a short explanation describing how timestamps, counters, or nonces can help prevent replay attacks. In your answer, explain that the receiver can check whether a message is old, duplicated, or out of sequence.

What to Submit

- Screenshot showing `message1.txt` restored to the original valid message.
- Screenshot showing execution of `python3 hmac_gen.py` and the generated HMAC tag.
- Screenshot showing successful verification of the original message.
- Screenshot showing the same message being accepted again when replayed.
- Screenshot showing the updated message with a timestamp or counter.
- Screenshot showing generation and verification of the new tag for the updated message.
- A short explanation describing the replay attack observed in your experiment.
- A short discussion explaining how timestamps, counters, or nonces can mitigate replay attacks.

4.9 Task 9: Implementing a PRF-Based Authentication Function

In the lecture, we discussed that **pseudorandom functions (PRFs)** are closely related to **Message Authentication Codes (MACs)**. A PRF is a keyed function that produces outputs that appear random to anyone who does not know the secret key. Because of this property, a secure PRF can be used to build a secure MAC.
In this task, you will **design and implement your own simple PRF-based authentication script** and use it to generate authentication tags for messages.

Step 1: Create a New Python Script
Create a new Python file in your working directory:

```
nano prf_experiment.py
```

Step 2: Design Your Own PRF Function
Inside the script, implement your own function that takes a **secret key** and a **message** as inputs and produces an authentication value.
Your function must satisfy the following:

- It must take two inputs: a secret key K and a message M.
- It must use a cryptographic hash function such as `SHA-256`.
- It must combine the key and the message before hashing.
- The output should be displayed as a hexadecimal authentication tag.

Mathematically, your function should follow a structure similar to

$$F_K(M) = H(K \| M)$$

where H represents a hash function.

Step 3: Generate Tags for Multiple Messages
Use your script to compute authentication values for at least three different messages, for example:

- Message 1
- Message 2
- Message 3

Display the message and its corresponding output value.

Step 4: Modify a Message
Change one of the messages slightly (for example, modify a word or number) and run your script again.
Observe how the authentication output changes.

Step 5: Change the Secret Key
Modify the secret key in your script and run the program again.
Observe how the outputs change even when the message remains the same.

Step 6: Analyze the Results
Based on your experiment, answer the following questions:

- What happens to the authentication output when the message changes?
- What happens when the secret key changes?
- Why would it be difficult for an attacker to compute the correct output without knowing the secret key?
- Why can a PRF be used to build a message authentication mechanism?

What to Submit

- Screenshot of your Python script.
- Screenshot showing the execution of your program.
- Screenshot showing results after modifying a message.
- Screenshot showing results after changing the key.
- A short explanation describing the relationship between PRFs and MACs based on your experiment.

4.10 Submission Requirements

- Submit a single **PDF report** on Canvas by the stated deadline. Only **one group member** should submit the report on behalf of the group.
- Your report must be clearly structured and organized by task (**Task 1 through Task 9**).
- For each task, your report must include:
  - A clear explanation of what was implemented or performed (written in complete sentences).
  - Screenshots showing execution in the **Kali Linux / Ubuntu terminal**.
  - Python code snippets where scripts were written.
  - Generated outputs such as **HMAC tags or PRF outputs**.
  - Interpretation of results (do not submit raw output without explanation).
- **Task-Specific Requirements:**

- **Task 1 (Lab Setup and Directory Creation):**
  * Screenshot showing creation of the lab working directory.
  * Screenshot confirming the current directory using `pwd` or `ls`.
- **Task 2 (Message Creation):**
  * Screenshots showing the creation of plaintext message files.
  * Evidence showing the contents of each message using `cat`.
- **Task 3 (HMAC Generation Script):**
  * Screenshot of your Python script used to generate HMAC tags.
  * Terminal output showing the generated authentication tag.
- **Task 4 (Message Verification):**
  * Screenshot showing successful verification of a valid message-tag pair.
  * Brief explanation of why the verification succeeds.
- **Task 5 (Message Tampering Experiment):**
  * Evidence showing modification of the message file.
  * Screenshot demonstrating verification failure when the message is altered.
  * Explanation describing why the MAC no longer verifies.
- **Task 6 (Multiple Message Authentication):**
  * Evidence showing HMAC generation for multiple messages.
  * Comparison of authentication tags produced for different messages.
- **Task 7 (Secret Key Change Experiment):**
  * Screenshot showing modification of the secret key in the script.
  * Output demonstrating how authentication tags change after the key is modified.
  * Short explanation of why the key affects authentication results.
- **Task 8 (Replay Attack Experiment):**
  * Screenshot showing verification of a valid message-tag pair.
  * Screenshot showing the same message accepted again when replayed.
  * Explanation describing how replay attacks work.
  * Discussion of mitigation techniques such as timestamps, counters, or nonces.
- **Task 9 (PRF-Based Authentication Script):**
  * Screenshot of your self-written Python script implementing a PRF-style function.
  * Evidence showing authentication outputs for multiple messages.
  * Evidence showing output changes when the message changes.
  * Evidence showing output changes when the key changes.
  * Short explanation describing the relationship between PRFs and MACs.

- Your Python source code must be included either:
  - as separate files attached to the submission, or
  - embedded within the report.
- All results must be **reproducible**. If the commands, scripts, or steps cannot be followed or verified, points may be reduced.
- Work that cannot be verified (for example missing screenshots, missing code, unclear reasoning, or undocumented parameters) may receive reduced points even if the final outputs appear correct.

Grading Rubric (Total: 100 Points)

Task	Assessment Criteria	Points
Task 1: Lab Setup and Directory Creation (10 pts)	Correct creation of lab working directory in Kali/Ubuntu	3
	Evidence of navigation and verification using terminal commands	3
	Screenshots clearly showing environment setup	2
	Brief explanation of the purpose of the working directory	2
Task 2: Message Creation (10 pts)	Creation of multiple plaintext message files	4
	Correct use of terminal commands to display message contents	3
	Screenshots demonstrating successful file creation	3
Task 3: HMAC Generation Script (15 pts)	Correct implementation of HMAC generation using Python	6
	Successful computation of authentication tag	4
	Screenshot showing execution of the script	3
	Explanation of how the tag is generated using a secret key	2

Task	Assessment Criteria	Points
Task 4: Message Verification (10 pts)	Correct verification process implemented in Python	4
	Successful validation of a correct message-tag pair	3
	Screenshots demonstrating verification success	3
Task 5: Message Tampering Experiment (10 pts)	Modification of the original message file	3
	Demonstration that verification fails after modification	4
	Explanation of why MAC verification fails for altered messages	3
Task 6: Multiple Message Authentication (10 pts)	Generation of HMAC tags for multiple messages	4
	Clear comparison of authentication tags across messages	3
	Explanation of why tags differ even when the same key is used	3
Task 7: Secret Key Change Experiment (10 pts)	Modification of the secret key in the script	3
	Demonstration that tag values change when the key changes	4
	Explanation of the role of the secret key in authentication	3
Task 8: Replay Attack Experiment (10 pts)	Successful demonstration of replay scenario using valid message-tag pair	4
	Evidence showing repeated acceptance of the same message	3
	Explanation of why MAC alone does not prevent replay	2

Task	Assessment Criteria	Points
	Discussion of mitigation techniques (timestamps, counters, nonces)	1
Task 9: PRF-Based Authentication Script (10 pts)	Student-developed PRF-style authentication script	4
	Demonstration of tag outputs for multiple messages	3
	Evidence of output change when message or key changes	2
	Explanation of PRF and MAC relationship	1
Report Quality (5 pts)	Clear organization and logical report structure	2
	Screenshots and outputs clearly documented	2
	Code readable and properly formatted	1
Total		**100**